

ReFlex: Feedback-Enhanced Constraint Repair for ReKep Motion Planning

Anonymous Authors

Abstract—ReKep 类机器人操作系统使用视觉语言推理生成分阶段任务约束，再由下游运动规划器求解这些约束。这一设计能够把语义任务描述连接到可执行机器人行为，但也引入了一个关键失效点：如果生成的约束不完整、过弱或与任务语义不一致，规划器仍可能求出一条形式上可行、但实际目标错误的轨迹。本文提出ReFlex，一个面向ReKep类运动规划的反馈增强约束修复框架。ReFlex 插入在规划与执行之间，重点处理 *constraint-induced errors*，即主要由阶段约束规格错误引起、而非由感知或底层求解器实现引起的规划失败。该框架在执行前检测经典轨迹症状，将其组织为结构化诊断协议 *RERS-v1* (ReFlex Error Report Standard v1)，进一步归因到可能的约束层原因，并把诊断结果转化为用于阶段约束重生成和重规划的修复提示。ReFlex 聚焦五类代表性错误：目标对象绑定错误、目标关键点或功能区域绑定错误、趋近方向错误、进展停滞错误和碰撞风险。在130个LIBERO任务上的评测中，ReFlex 的自然任务成功率为80.5%，较原始ReKep 提高29.9个百分点；约束失配评测成功率为83.4%，检测F1 为91.0%。本文进一步通过五类错误的正确/错误分阶段约束方程组和MuJoCo 经典场景定性对照，分析约束规格差异及局部修复范围。

I. 引言

近年来，视觉语言模型 (VLM)、大语言模型 (LLM) 和视觉语言动作模型 (VLA) 的发展，使机器人能够用更高层的多模态推理来理解操作任务。其中，ReKep 类方法具有代表性：它不直接从语言预测动作，而是先从场景图像中提取语义关键点，再由VLM 生成分阶段的约束函数，最后交给下游优化式运动规划器求解轨迹。这样，自然语言指令和视觉观测被转化为结构化约束，机器人再依据这些约束生成末端执行器运动。

但是，这一优势也暴露了一个核心风险：一旦VLM 生成的阶段约束错误、不完整或与任务语义错位，优化器仍然可能成功求解，只是求解的是一个被错误表述的问题。换句话说，规划器通常只会求解“给定给它的问题”，而不保证这个问题就是“本来应该被提出的问题”。这造成了 *constraint solvability* 与 *task correctness* 之间的错位。一条轨迹可能平滑、可达、甚至看起来可执行，但仍然可能接近错误物体、绑定到错误功能区域、沿无效方向运动，或进入碰撞风险区域。

传统运动规划保护机制并不能完全覆盖这一问题。碰撞代价、IK 可达性、路径平滑等项主要处理几何和物理可行性，它们通常不回答以下问题：机器人是否正在接近正确语义目标？当前阶段是否确实在推进任务完成？当前VLM 生成的约束是否应该在执行前被修订？在ReKep类系统中，这些问题尤其关键，因为阶段约束并不是人工给定的确定真值，而是由VLM 生成的中间规格，本身就可能成为错误来源。

本文的基本观点是：约束生成不应被视为一次性过程，而应与显式执行前反馈机制结合。系统应在执行前检查候选轨迹，识别经典失败症状，将这些症状归因

到可能的约束层缺陷，并把结构化修复信息反馈给阶段约束生成模块。基于这一思想，我们提出ReFlex，一个建立在ReKep 类规划框架之上的反馈增强约束修复方法。ReFlex 不替代原有规划器，而是在规划后、执行前分析轨迹，生成结构化错误报告，定位可能的阶段约束问题，并构造修复prompt，引导约束重生成和重规划。

当前版本的ReFlex 有意限定在一个清晰范围内：*constraint-induced errors*。本文不试图一次性解决所有机器人操作失败来源，尤其不把感知错误、关键点提取失败或底层求解器数值不稳定作为主要修复对象。我们关注的问题是：当阶段约束本身导致规划行为错误时，能否在执行前识别其轨迹症状，并以结构化方式修复约束规格？这一边界使ReFlex 能够作用在ReKep 管线中最具语义意义的控制点，即VLM 生成的 *subgoal constraints* 与 *path constraints*。

在这一范围内，ReFlex 关注五类代表性错误：目标对象绑定错误、目标关键点或功能区域绑定错误、趋近方向错误、进展停滞错误和碰撞风险。这五类错误覆盖了运动规划质量的三个互补层面：机器人是否追求正确语义目标，轨迹是否有效推进目标，以及候选路径是否具有物理执行风险。ReFlex 进一步用 *RERS-v1* (ReFlex Error Report Standard v1) 表示这些错误。RERS-v1 将症状、证据、严重程度、约束层归因和修复目标组织为机器可读的结构化报告，从而支持后续约束重生成，而不是依赖临时人工分析或自由文本反馈。

因此，本文目标不是单纯检测坏轨迹，而是闭合 *constraint generation*、*trajectory analysis* 与 *constraint repair* 之间的反馈环。通过把失败现象显式化、结构化和可操作化，ReFlex 将执行前检查转化为对上游运动规格的主动修复机制。

为使约束规格成为可核查的论述对象，本文为五类症状给出正确与错误的分阶段方程组，并配套基于ReFlex 错误体系的MuJoCo 经典场景定性对照，解释约束内容与任务行为之间的关系。LIBERO 定量评测则从任务成功率、检测、归因、修复和成本等方面评价方法效果。

本文主要贡献如下：

- 1) 我们将约束诱发的运动规划错误定义为ReKep 类VLM 引导规划管线中的一个独立且重要的失败模式。
- 2) 我们提出ReFlex，一个执行前反馈框架，能够检测经典规划失败症状，将其归因到可能的阶段约束缺陷，并把修复信号反馈给约束生成过程。
- 3) 我们定义RERS-v1，一个结构化错误报告标准，用于形式化症状证据、归因输出和面向约束重生成的修复接口。
- 4) 我们给出一个连接轨迹分析、结构化报告、prompt 驱动约束修订和重规划的修复闭环，为更可靠的VLM 引导机器人运动规划提供具体实现路径。

5) 我们在130个LIBERO任务上开展系统比较、消融与成本分析，并提供五类经典错误的成对约束规格、MuJoCo关键帧及进展时序，分析方法效果与局部修复范围。

总体而言，ReFlex是ReKep类规划系统中的执行前安全与修复层。它不替代原有语义规划机制，而是通过检查候选行为并修订产生该行为的约束，增强原系统的可靠性和可解释性。

II. 相关工作

与本文最相关的研究可以概括为三条线索：用于机器人规划与语义grounding的基础模型，机器人执行中的闭环反馈与重规划，以及大模型输出的结构化自修正机制。ReFlex位于这三类工作的交叉处。它继承ReKep的约束生成思想，但重点不是重新设计低层运动规划器，而是把执行前发现的问题转化为结构化、可修复的上游约束反馈。

A. 用于机器人规划与语义Grounding的基础模型

近年来，LLM、VLM和VLA系统显著降低了机器人任务规划中的人工工程负担。SayCan展示了语言模型与affordance估计结合后可用于高层技能选择；Code as Policies表明代码生成可以作为自然语言推理与机器人控制API之间的中间接口；VoxPoser则进一步强调了语言、视觉和三维空间表示结合后，可以把高层任务意图转化为可执行的几何目标。

后续方法更加关注语义推理与几何精度之间的接口。PaLM-E等具身多模态模型扩展了基础模型在机器人推理中的作用，RAM、HAMSTER等工作引入更显式的对象中心或层级表示，以缩小语义指令与运动执行之间的差距。ReKep在这一方向中具有特殊意义：它不直接预测动作，而是让VLM生成阶段性的subgoal constraints和path constraints，再交由优化式规划器求解。这一设计保留了经典规划的可解释性和约束求解能力，同时把基础模型用作任务规格生成器。ReFlex正是在这个接口上工作，关注这些生成约束不完整、过弱或语义错位时应如何诊断和修复。

B. 机器人闭环反馈、反思与重规划

另一类相关工作研究机器人如何利用执行反馈从规划或执行失败中恢复。该方向的核心认识是：仅依赖语言生成的开放环动作序列通常比较脆弱，符号层面看似合理的动作序列，在真实几何约束、物理动态或任务状态变化下可能失败。Inner Monologue将文本化执行反馈引入语言规划；BrainBody-LLM、Goal2Skill等系统进一步区分高层推理与低层执行，并把执行结果、仿真错误或后置条件检查反馈给规划过程。

这些工作支持了ReFlex的反馈视角。不过，已有机器人反馈方法通常在执行失败或在线监测事件发生后，修订动作序列、任务分解或子目标结构。ReFlex关注的是一个更窄但互补的问题：阶段约束生成过程自身引入的错误。它强调的问题是“上游运动规格是否一开始就被错误表述，以及如何在执行前修复这个规格”。因此，ReFlex是一个作用在约束层的执行前反思层，而不仅是动作序列层面的恢复机制。

C. 结构化自修正与外部反馈修复

第三条相关线索来自大模型自修正和反馈驱动修订研究。已有工作表明，大模型可以通过critique、verification和refinement改善输出，尤其是在反馈信息明确、修正目标受限、且中间表示结构化时效果更稳定。CRITIC等工具交互方法说明，外部验证器通常比无约束自我反思更可靠；同时，也有研究指出，自修正能否成功高度依赖反馈通道是否具体、目标是否边界清晰、输出契约是否可检查。

这一点直接影响ReFlex的repair阶段。ReFlex不依赖自由形式的prompt重写，而是先通过RERS-v1将轨迹症状组织成结构化诊断，再把诊断映射到约束层归因，最后基于明确证据、受限修复目标和结构化输出契约构造修复prompt。换言之，ReFlex采用了大模型修正研究中的一个基本原则：只有当模型拿到明确证据、局部编辑目标和输出约束时，修订才更可能稳定。

D. ReFlex的定位

综合来看，ReFlex的定位可以概括为：从基础模型机器人方向继承“语言和视觉模型生成任务相关运动规格”的观点；从机器人闭环反馈方向继承“失败应反馈到上游推理而不是只在下游处理”的观点；从大模型自修正方向继承“修复需要结构化证据和受限编辑目标”的原则。ReFlex的核心贡献，是把这些思想整合成一个执行前框架：检测经典运动规划症状，将其组织为结构化报告，归因到可能的约束缺陷，并反馈给阶段约束生成器进行修复和重规划。

III. 方法

A. 问题建模

本文考虑一个ReKep类机器人操作设置：系统先根据任务指令和场景观测生成阶段运动约束，再由下游规划器求解候选轨迹。令多模态任务上下文表示为

$$\mathcal{X} = (I, O, K, c),$$

其中 I 表示自然语言指令， O 表示场景观测， K 表示语义关键点或任务相关锚点集合， c 表示当前机器人状态和环境上下文。基于 $\mathcal{X}$ ，VLM生成一组阶段约束：

$$\mathcal{C} = \{(g_1, h_1), (g_2, h_2), \dots, (g_N, h_N)\},$$

其中 g_i 是第 i 个阶段的subgoal constraints， h_i 是该阶段执行过程中应持续满足的path constraints。随后，规划器求解这些约束并输出轨迹

$$\tau = \{a_1, a_2, \dots, a_T\},$$

每个动作写作

$$a_t = [x_t, y_t, z_t, q_t^x, q_t^y, q_t^z, q_t^w, u_t], \quad (1)$$

其中前三维为末端执行器位置，中间四维为四元数姿态， u_t 为夹爪命令。

ReFlex的关键观察是：该管线中的失败并不一定来自规划器本身。规划器可能生成几何上可行的轨迹，但它所求解的阶段目标已经被上游VLM错误表述。本文因此聚焦constraint-induced motion planning errors：主因来自阶

段约束规格的错误，而不是感知、关键点提取或底层求解器实现缺陷。

在这一设定下，ReFlex 的输入是候选轨迹、当前阶段上下文和对应阶段约束，输出不是简单二分类，而是结构化报告、约束层归因和修复信号：

$$(\tau, \mathcal{X}, g_i, h_i) \rightarrow (r_i, a_i, p_i), \quad (2)$$

其中 r_i 是结构化错误报告， a_i 是约束层归因， p_i 是反馈给 ReKep 约束生成器的修复信号。ReFlex 因此把执行前风险视为上游问题表述的反馈，而不只是下游可行性问题。

B. 错误体系与约束规格对照

ReFlex v1 考虑五类轨迹症状，分别属于语义一致性、目标趋近和物理执行风险。为了把可见动作接回约束生成接口，本文对每类案例同时给出任务要求、正确约束规格和错误约束规格。令阶段约束写为

$$\mathcal{C}_i^\pm = \mathcal{C}_{i,sg}^\pm \cup \mathcal{C}_{i,path}^\pm, \quad (3)$$

其中上标+ 和- 分别表示正确与错误规格，sg 表示阶段终点条件，path 表示阶段过程中应持续满足的条件；两者分别对应问题建模中的 g_i 与 h_i 。约束可以写为等式或带容差的不等式；这里的 T 是所分析阶段的终点，不必是整个视频的最后一帧。

关键区别是独立任务要求与生成规格并不总是一致。若任务要求记为 $\mathcal{R}_i$ ，则一个错误候选可能具有如下性质：

$$\tau^- \models \mathcal{C}_i^-, \quad \tau^- \not\models \mathcal{R}_i. \quad (4)$$

因此，仅检查候选是否满足原始生成约束，会漏掉对象引用错误、功能区域错位以及必要条件缺失。ReFlex 需要结合独立任务上下文识别这些症状，再反馈应修订的约束内容。

图 1 使用 MuJoCo 中的 Panda 与生活物体展示五组正确/错误动作，表 I 列出各组最关键的约束差异。 $p_e(t)$ 为 TCP 位置， k_o 为对象或功能区域的抓取参考点， k_G 为搬运阶段的放置参考点， $d_G(t) = \|p_e(t) - k_G\|$ 。 $v_e(t)$ 为最后接近段的非零运动方向， $n_\downarrow$ 和 $n_\rightarrow$ 分别为向下与水平单位方向。 $\mathcal{B}(t)$ 表示相关机器人和被持物体几何， $\mathcal{O}$ 表示禁止接触的障碍物。

a) 语义一致性。：Scene A 的对象绑定案例中，任务始终要求抓苹果，而错误规格把抓取参考点替换为梨。Scene B 中，两组对象均为锅铲，但功能区域分别绑定到手柄和铲面。两类错误都涉及 subgoal 的引用内容；感知或关键点提取模块本身的错误不在当前修复范围内。

b) 目标趋近。：Scene C 保留相同的按钮目标点和夹爪朝向，只改变最后接近段的方向关系。Scene A 的搬运案例则说明另一种缺陷：保持抓取、姿态和可达性，并不足以规定何时完成当前阶段。缺少有效终点条件的规格允许局部往复，但这种遗漏并不必然产生停滞；检测仍需观察轨迹中的实际进展。绕障、等待和阶段结束后的静止不自动构成该症状。

c) 物理执行风险。：Scene D 的正确搬运通过升高苹果越过挡板；错误参考沿低位朝目标移动，完整候选路线与挡板相交。安全要求作用于机械臂及被持苹果，而非只作用于 TCP。演示的错误播放在苹果距挡板约 2.5 mm 时停止，虚线展示未播放的危险后续路线；正确播放最终把苹果放稳、松爪并抬离。

d) 案例的证据范围。：这些方程组用于表达经典错误场景中的任务规格差异，与 MuJoCo 定性对照共同解释局部修复应修改和保留的内容。正确列为任务参考行为，定性对照本身不用于统计自动修复成功率。五类症状可以共存，其完整阶段定义、符号及对应关系见附录 A。

C. 设计动机：为什么已有处理不够

现有 ReKep 类系统和经典规划管线在给定约束后能够有效求解轨迹，但通常默认该约束集本身是正确的。因此，它们更关注当前约束是否可解、路径是否碰撞、失败后能否重试，却较少直接回答：阶段约束是否把任务语义表述错了？若表述错了，应该如何记录、归因并反馈给约束生成器？表 II 总结了这一差距。

D. ReFlex 总体框架

ReFlex 插入在约束驱动规划和真实执行之间。对需要修复的阶段，其反馈过程为

$$\begin{aligned} &(\tau, \mathcal{X}, g_i, h_i) \xrightarrow{\text{ReFlex}} (r_i, \mathcal{A}_i, \mathcal{P}_i), \\ &(\mathcal{X}, g_i, h_i, r_i, \mathcal{A}_i, \mathcal{P}_i) \xrightarrow{\text{ReKep generator}} (\hat{g}_i, \hat{h}_i), \\ &(\hat{g}_i, \hat{h}_i, \mathcal{X}) \xrightarrow{\text{planner}} \hat{\tau} \xrightarrow{\text{ReFlex}} \text{re-check}. \end{aligned} \quad (5)$$

其中 r_i 记录诊断证据， $\mathcal{A}_i$ 表示归因结果， $\mathcal{P}_i$ 表示修复规范。ReKep 仍负责生成新约束；ReFlex 负责检查、报告、归因和反馈。若初始轨迹或修复后轨迹通过检查，则可进入执行；若预算耗尽仍未通过，则拒绝执行。

图 2 展示了该架构。上半部分是 ReKep 主链路：场景观测、关键点、VLM 约束生成、规划器求解。ReFlex 从候选 motion array 之后开始工作：先归一化轨迹并提取上下文特征，再执行症状检测，随后生成 RERS-v1 报告、进行约束归因，并构造修复 prompt。反馈箭头回到约束生成，而不是回到关键点提取或直接修改轨迹。

E. 症状特征提取与检测

ReFlex 将可观测规划失败建模为轨迹症状。对轨迹 $\tau = \{a_t\}_{t=1}^T$ ，系统先拆分末端位置 $p_t \in \mathbb{R}^3$ 、四元数姿态 q_t 和夹爪命令，并结合当前阶段的目标对象、目标关键点、功能区域和障碍几何信息，提取三类特征。

第一类是语义一致性特征，包括末端到目标对象中心、目标关键点、干扰对象和干扰关键点的距离，以及目标与干扰物之间的距离 margin。这类特征用于识别目标对象绑定错误和关键点/区域绑定错误。

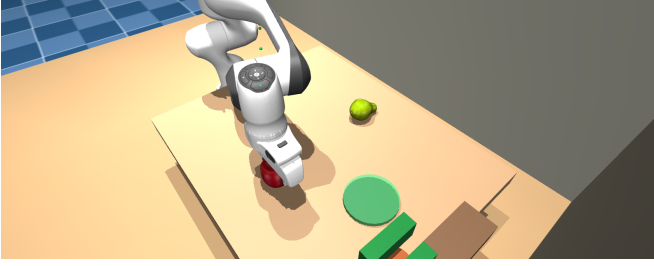
第二类是目标趋近特征。令 d_t 表示末端到当前目标的距离，ReFlex 计算 $d_t - d_{t-1}$ 、归一化进展趋势、运动方向与目标方向之间的投影等信号。这些特征用于判断轨迹是否在有效接近目标，支持趋近方向错误和进展停滞错误检测。

第三类是物理执行特征。当前版本主要使用 signed distance、碰撞代价和逐步最小 obstacle clearance，判断轨迹是否进入碰撞或低间隙风险区域。

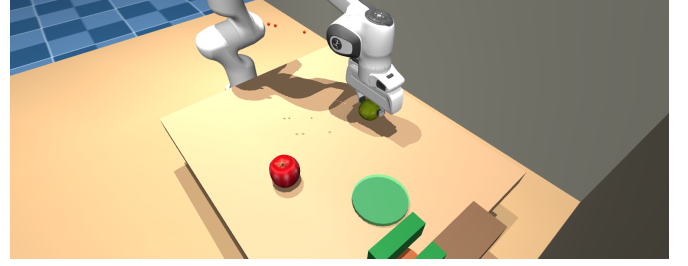
正确约束与参考动作

错误约束与错误动作

(a) 对象绑定: 抓取苹果

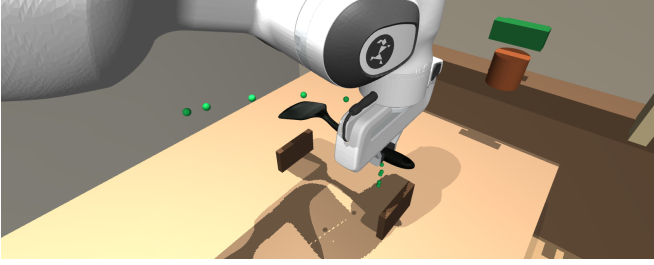


$$\|p_e(T) - k_{\text{apple}}\| \leq \epsilon_p$$

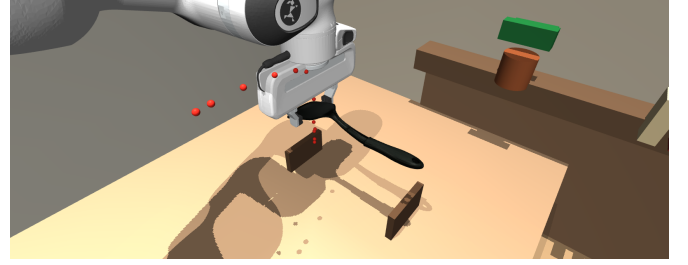


$$\|p_e(T) - k_{\text{pear}}\| \leq \epsilon_p$$

(b) 功能区域: 抓住锅铲手柄

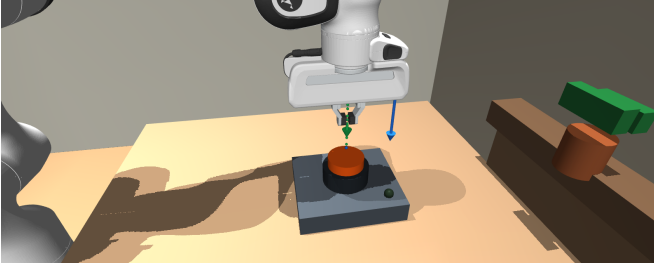


$$\|p_e(T) - k_{\text{handle}}\| \leq \epsilon_p$$

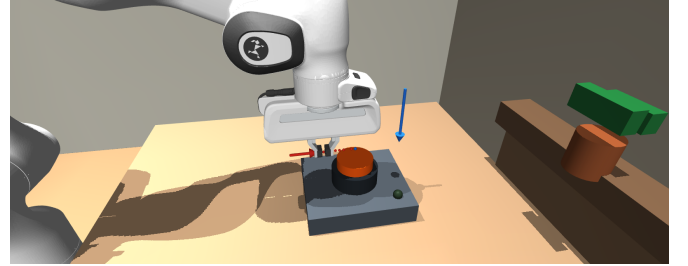


$$\|p_e(T) - k_{\text{blade}}\| \leq \epsilon_p$$

(c) 趋近方向: 从上方接近按钮

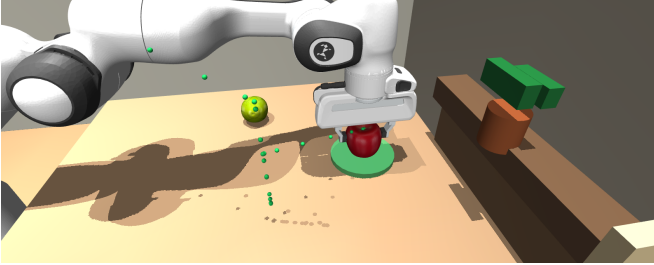


$$v_e \cdot n_{\downarrow} \geq \|v_e\| \cos \theta$$

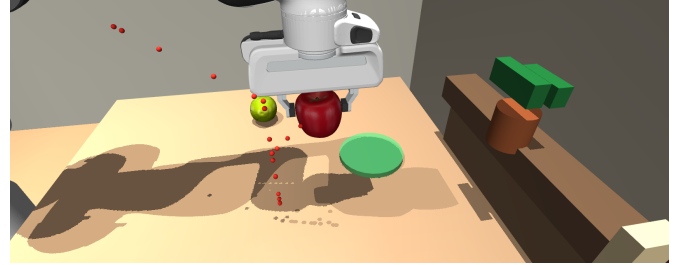


$$v_e \cdot n_{\rightarrow} \geq \|v_e\| \cos \theta$$

(d) 进展停滞: 把苹果搬到目标区域

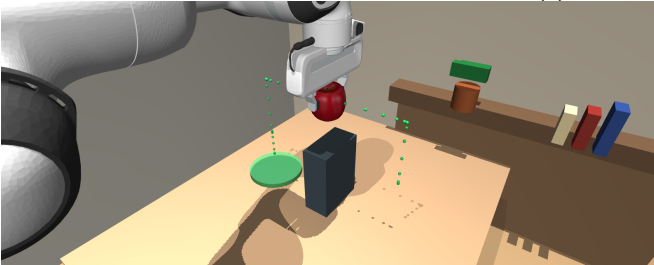


$$d_G(T) \leq \epsilon_p$$

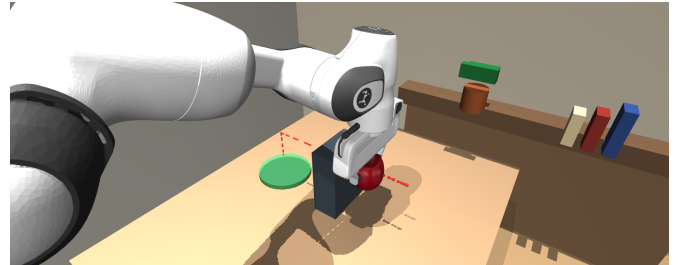


$$\mathcal{C}_{\text{sg}}^- = \emptyset$$

(e) 碰撞风险: 把苹果搬过挡板



$$\text{dist}(B(t), \mathcal{O}) \geq \delta$$



缺失路径安全间隙条件

Fig. 1. 基于ReFlex 错误体系的五类经典场景MuJoCo 定性对照。各行展示相关阶段的关键约束差异，完整方程组见附录 A。两列共享场景与相机，正确列表示任务要求下的参考行为。(c) 取接近中段；(d) 的停滞需结合图 8 的时序证据；(e) 红色虚线为未播放的危险候选路线，错误行为为停在接触前。

TABLE I
五类案例的关键约束差异与候选修复目标。表中仅突出区分两组行为的条件，共同条件及分阶段方程组见附录 A。

症状与任务	正确规格 C^+	错误规格 C^-	约束层修复目标
(a) 对象绑定: 抓苹果	$\ p_e(T) - k_{\text{apple}}\ \leq \epsilon_p$	$\ p_e(T) - k_{\text{pear}}\ \leq \epsilon_p$	将当前抓取阶段的对象引用从梨改为苹果。
(b) 区域绑定: 抓锅铲手柄	$\ p_e(T) - k_{\text{handle}}\ \leq \epsilon_p$	$\ p_e(T) - k_{\text{blade}}\ \leq \epsilon_p$	保留锅铲对象，修订功能区域引用。
(c) 方向: 从上方接近按钮	相同终点, 且 $v_e \cdot n_{\downarrow} \geq \ v_e\ \cos \theta$	相同终点, 但 $v_e \cdot n_{\rightarrow} \geq \ v_e\ \cos \theta$	修订最后接近段的方向关系, 保留目标点。
(d) 停滞: 搬运苹果	$d_G(T) \leq \epsilon_p$, 保持抓取与姿态	保留抓取与姿态, 但省略搬运阶段终点要求	补齐阶段成功条件; 推进证据按阶段解释。
(e) 碰撞: 越过挡板	保留放置目标, 并要求 $\text{dist}(\mathcal{B}(t), \mathcal{O}) \geq \delta$	保留放置目标及低位搬运, 却省略安全间隙条件	为搬运路径补充机器人及被持苹果的避障要求。

TABLE II
已有处理策略的局限及ReFlex 的对应设计。

已有处理方式	局限	ReFlex 设计响应
Planner-centric feasibility checking	默认当前阶段约束正确, 只检查是否能找到可行轨迹。	显式检查规划器是否在求解错误的阶段目标。
Downstream safety or collision checking	能发现不安全路径, 但通常不解释上游语义或几何约束哪里有问题。	将轨迹症状连接到可能的约束层原因。
Scalar anomaly scoring or ad hoc failure flags	只能给出可疑分数或失败标记, 难以用于日志、归因和修复。	用RERS-v1 记录结构化证据、严重度和修复接口。
Free-form diagnosis and retry prompting	自由文本修复范围难以控制, 容易改动无关阶段或引入新目标。	使用阶段局部、归因驱动、带preservation rules 的prompt。
One-shot rejection without upstream revision	只能阻止执行, 不能修正产生错误轨迹的约束规格。	将诊断反馈给约束生成器, 重生成约束并复检。

给定特征集合 $F(\tau, \mathcal{X})$, ReFlex 使用一组确定性规则检测器

$$\mathcal{D} = \{D_1, D_2, \dots, D_M\}.$$

每个检测器对应一个经典错误类型, 并输出零个或多个症状事件。选择规则式检测是为了保证证据可复现、阈值可追踪、报告可解释, 而不是把错误判断压缩为黑箱异常分数。

F. RERS-v1 结构化错误报告

检测到的症状会被组织为RERS-v1。其目标是把低层detector 输出转化为统一、机器可读、可用于归因和修复的中间表示。对阶段 i , 报告可写为

$$R_i = (\text{meta}_i, \text{decision}_i, \mathcal{E}_i, \mathcal{A}_i, \mathcal{P}_i), \quad (6)$$

其中 meta_i 保存任务、阶段、轨迹和版本等元信息, $\text{decision}_i \in \{\text{PASS}, \text{WARN}, \text{FAIL}\}$ 是阶段级判断, $\mathcal{E}_i$ 是症状事件集合, $\mathcal{A}_i$ 是约束层归因集合, $\mathcal{P}_i$ 是面向约束重生成的修复规范。

每个症状事件可写为

$$e_{ij} = (\text{type}_{ij}, \text{severity}_{ij}, \text{confidence}_{ij}, \text{time_range}_{ij}, \text{rule_id}_{ij}, \text{evidence}_{ij}, \text{explanation}_{ij}). \quad (7)$$

其中 evidence_{ij} 不以自由文本保存, 而是记录特征名、观测值、阈值、比较关系和聚合方式。这样, 同一份报告既可用于在线控制, 也可用于离线复盘、统计评估和prompt 构造。

表 IV 以图 1(a) 的苹果/梨差异说明报告字段与对象绑定症状的对应关系。该轨迹可能几何上平滑且可执行, 但语义上接近了错误对象。RERS-v1 将这一现象组织为事件、证据、归因和修复目标, 从而形成从症状到修复的接口:

$$\mathcal{E}_i \rightarrow \mathcal{A}_i \rightarrow \mathcal{P}_i.$$

G. 约束归因

在症状被组织为RERS-v1 后, ReFlex 执行约束归因。其目标是把轨迹层症状映射为当前阶段约束中更可能存在的规格缺陷。形式上, 我们定义归因映射

$$\phi: \mathcal{E}_i \rightarrow \mathcal{A}_i,$$

将一个或多个症状事件转换为一个或多个约束层原因假设。ReFlex v1 的归因类型包括目标对象描述不匹配、目标关键点或功能区域绑定不匹配、趋近方向或相对几何关系错误、阶段成功条件过弱, 以及缺失碰撞规避或安全间隙约束。

该阶段采用规则引导, 而不是完全开放式推理。原因在于ReFlex 的输出需要稳定、可解释并可直接指导修复。例如, 目标对象绑定错误通常归因到当前阶段subgoal 中的对象描述不匹配; 碰撞风险通常归因到path constraint 中缺少clearance 或避障要求。agent 可以用于补充解释或排序候选原因, 但核心归因应始终由结构化证据和固定schema 约束。

表 I 的规格对照为局部编辑提供了具体模板: 替换对象或区域引用、修订接近方向、补齐阶段终点条件, 以及补充路径安全间隙。归因仍需结合实际候选约束核查, 不能仅凭一张错误截图认定唯一原因。

ReFlex over ReKep: Closed-Loop Constraint Error Detection and Repair

Case example: correcting a wrong target-object trajectory before execution

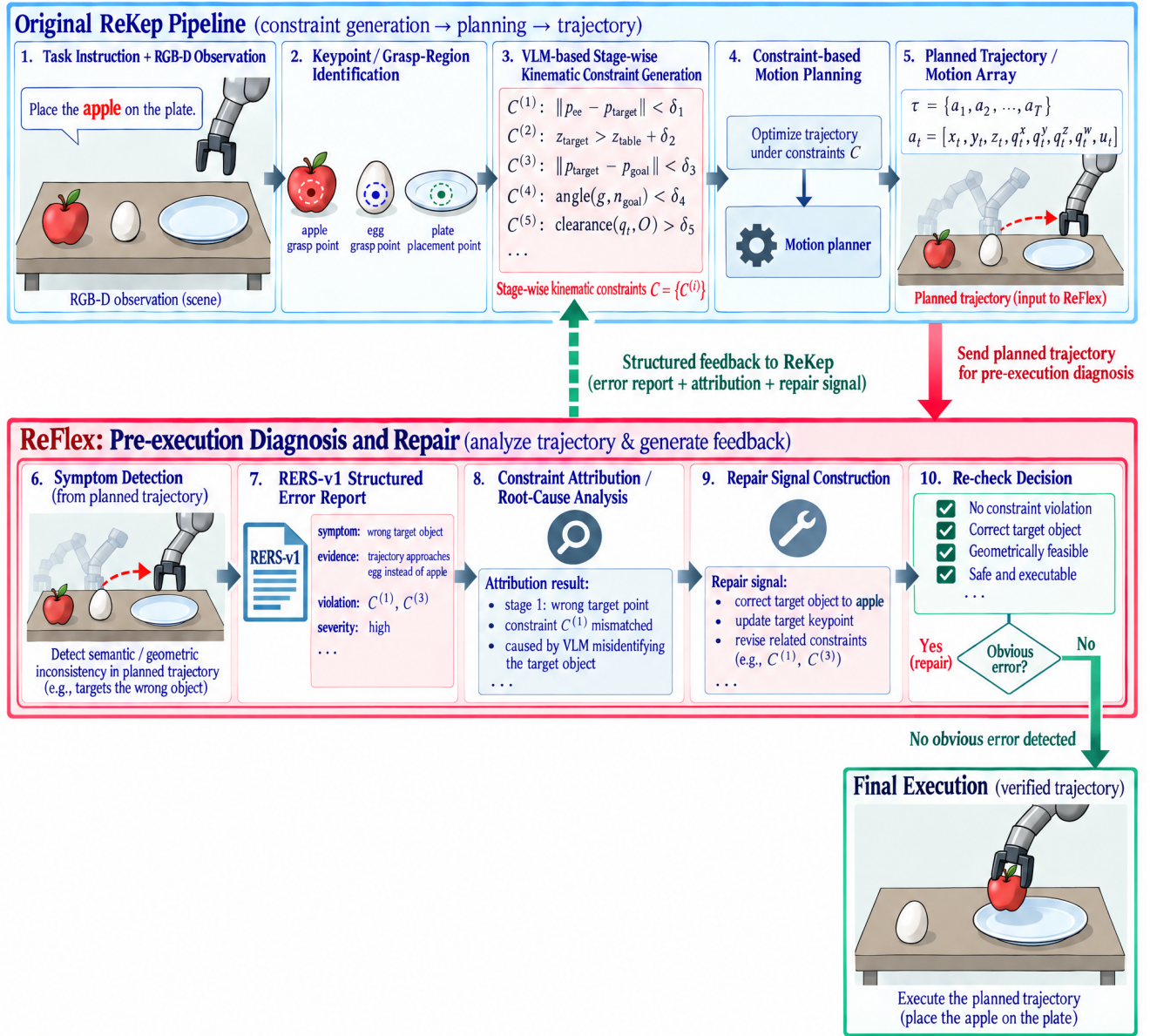


Fig. 2. 建立在ReKep 之上的ReFlex 总体框架。轨迹生成后、执行前，ReFlex 产生结构化证据、归因和修复反馈；ReKep 使用该反馈重生成阶段约束并重新规划，每条新轨迹都会再次进入相同检查流程。

H. 修复Prompt 构造

完成归因后，ReFlex 需要把诊断结果转化为可驱动约束重生成的prompt。该prompt 不是自由重写请求，而是一个受限的局部修复包，包含五部分：任务定义、当前阶段上下文、RERS-v1 结构化报告、少量修复示例和明确输出契约。

具体来说，prompt 包含原始任务指令、失败阶段ID、原阶段约束、症状事件、约束归因、修复目标、必须保留内容、允许修改范围和禁止操作。其基本原则是：只修改当前失败阶段；保留未被报告指向的正确

约束；不引入新目标、新对象或无关任务意图；修复后的输出应显式区分repaired_subgoal_constraints与repaired_path_constraints。完整prompt 包见附录B。

以锅铲区域错误为例，Must Modify 指向抓取参考点及对应抓取几何，Must Preserve 保留锅铲对象、抬升目标和其他阶段；以停滞案例为例，只补齐搬运阶段的完成条件，并保留苹果绑定与抓取保持。附录A 的成对方程组说明编辑范围，附录B 的五类模式则把这一范围转化为修复提示。

TABLE III
RERS-v1 的概念层级。缩进节点表示包含关系；表中字段路径用于说明信息组织。

Schema 节点	父节点	说明
Root Report Object		
report	–	单个任务阶段的一份诊断对象。
meta	report	任务ID、阶段ID、轨迹ID、报告ID 和checker 版本。
decision	report	阶段级判断: PASS 、 WARN 或 FAIL 。
summary	report	事件数量、最高严重度和主要失败模式等摘要。
events[]	report	检测到的症状事件列表。
attributions[]	report	由症状推断出的约束层原因假设。
repair_spec	report	用于构造重生成prompt 的修复规格。
Nested Symptom Event Object: events[j]		
type	events[j]	症状类别, 例如object binding 或collision risk。
severity	events[j]	事件严重程度。
confidence	events[j]	detector 对当前事件成立的置信度。
time_range	events[j]	症状所在轨迹区间。
rule_id	events[j]	触发事件的规则编号。
evidence[]	events[j]	观测值、阈值和聚合方式等结构化证据。
explanation	events[j]	用于调试和prompt 构造的简短解释。
Nested Attribution Item: attributions[k]		
cause_type	attributions[k]	候选约束层原因。
target_scope	attributions[k]	指出原因关联subgoal、path 或两者。
confidence	attributions[k]	归因置信度。
rationale	attributions[k]	连接症状与原因的诊断解释。
Nested Repair Specification: repair_spec		
repair_target	repair_spec	待修复对象, 如对象引用、关键点绑定、趋近几何或clearance 规则。
repair_scope	repair_spec	修复范围, 例如仅修改当前阶段。
instruction	repair_spec	后续转化为重生成prompt 的修复指令。

TABLE IV
SCENE A 对象绑定症状与RERS-v1 报告字段的对应示例。

字段路径	示例值
report.meta.task_id	grasp_apple_from_table
report.meta.stage_id	stage_2_descend_to_grasp
report.decision	FAIL
report.summary.primary_failure_mode	target_object_binding_error
report.events[0].type	target_object_binding_error
report.events[0].evidence[0]	$m = \text{mean}_{t \in \mathcal{I}_a} (\ p_e(t) - k_{\text{pear}}\ - \ p_e(t) - k_{\text{apple}}\) < 0$; m 为接近窗口内干扰对象与目标对象距离的平均差。
report.events[0].explanation	末端轨迹在接近窗口内更靠近干扰梨, 而不是任务指定苹果。
report.attributions[0].cause_type	target_object_description_mismatch
report.attributions[0].target_scope	subgoal_constraint
report.repair_spec.repair_target	object_reference
report.repair_spec.instruction	将第2 阶段subgoal 的对象引用改为苹果抓取参考点; 保留下降、夹爪姿态与闭合条件。

I. 约束重生成与重规划闭环

如图 2 所示, 结构化诊断最终反馈给ReKep 的约束生成模块。令 R_i 为阶段 i 的报告, Q_i 为由其证据、归因和修复规格构造的prompt。约束生成器产生修订后的阶段约束:

$$(\hat{g}_i, \hat{h}_i) = \Psi(\mathcal{X}, g_i, h_i, Q_i), \quad (8)$$

其中 Ψ 表示被诊断prompt 条件化的上游约束生成过程。随后规划器在修订约束下重新求解:

$$\hat{\tau} = \Pi(\hat{g}_i, \hat{h}_i, \mathcal{X}), \quad (9)$$

并将新轨迹再次送入ReFlex 复检。完整闭环为

$$\begin{aligned} \tau &\rightarrow F(\tau, \mathcal{X}) \rightarrow \mathcal{E}_i \rightarrow R_i \rightarrow Q_i \\ &\xrightarrow{\text{ReKep generator}} (\hat{g}_i, \hat{h}_i) \rightarrow \hat{\tau} \rightarrow \text{re-check}. \end{aligned} \quad (10)$$

若轨迹得到**PASS**, 则允许进入执行; 若为**WARN** 或**FAIL**, 则在预算允许时继续修复, 否则拒绝执行。这

里的**PASS** 只表示在当前配置检查下没有发现阻塞性症状, 并不等价于对任务成功或物理安全的全局证明。

该闭环的一个重要特点是所有中间产物都可持久化: 结构化报告、修复prompt、重生成约束、重规划轨迹和复检结果都能通过report metadata 关联到具体任务、阶段和轨迹。这使ReFlex 不只是一个执行前筛查器, 而是一个可复盘、可统计、可迭代改进的约束修复机制。

IV. 实验与结果分析

本章在LIBERO 任务上比较ReFlex 与各类反馈基线, 分析检测、归因、修复、任务成功率以及计算成本。随后, 基于ReFlex 的五类错误体系, 在MuJoCo 经典场景中进行正确与错误行为的定性对照, 解释约束规格差异及相应修复目标。

A. 实验设置与评测协议

我们在全部**130 个LIBERO 任务**上评估ReFlex。底层环境采用MuJoCo/robosuite, 任务集合包括LIBERO-

Spatial、LIBERO-Object、LIBERO-Goal 各10 个任务，以及LIBERO-100 中的全部100 个任务。为了避免重复统计，本文将LIBERO-100 拆分为两个不重叠组：**LIBERO-Long** (libero_10, 10 个任务) 和**LIBERO-90** (90 个任务)。

任务组差异。 根据LIBERO 原始定义，Spatial、Object 和Goal 分别隔离不同的知识迁移因素。Spatial 中对对象集合相同，任务通常要求根据空间关系选择目标，例如把指定位置的碗放到盘子上。Object 中任务形式相近，但目标物体类型变化，用于考察对象绑定能力。Goal 中物体和空间关系基本固定，但目标状态变化，用于考察阶段目标和完成条件是否被正确表达。LIBERO-100 则包含更复杂的物体交互和多阶段组合，其中LIBERO-90 扩大任务多样性，Long 更关注长时程任务中的跨阶段一致性和错误累积。

与ReFlex 的关系。 ReFlex 五类错误按照具体任务阶段独立标注。Spatial 和Object 可以帮助考察空间指代和目标对象绑定；Goal 有助于检查目标状态和阶段完成条件；Long 和LIBERO-90 用于观察更复杂任务中的约束生成与修复适用范围。目标关键点、趋近方向、进展停滞和碰撞风险等症状则结合阶段目标、轨迹证据和任务语义共同判定。

系统设置保留LIBERO 的Franka Panda embodiment、原始语言指令和任务成功判据。ReKep 与ReFlex 在匹配比较中共享相同的感知、关键点和规划器配置。若ReFlex 诊断额外使用仿真几何或语义标注，则在公平比较中为相应基线提供同等输入。仿真器版本、控制器、相机设置、任务horizon、模型版本和修复预算在评测协议中固定。

闭环评测包含两个部分。第一部分是自然任务评测，用于比较原始ReKep、强反馈基线和完整ReFlex 的端到端任务成功率。第二部分是约束失配分析：围绕适用任务阶段中的典型约束失配现象，分别评估ReFlex 对五类错误模式的检测、归因和修复能力。该协议是本项目自定义评测层，用于补充LIBERO 原始任务成功判据。本章末尾另给出经典错误场景的定性对照。

我们区分完整任务rollout 和阶段级诊断实例。任务成功率以LIBERO 原始任务谓词为准，生成失败、规划失败、拒绝执行和预算耗尽都保留在完整rollout 分母中。诊断指标则在独立标注的候选轨迹上统计，并报告候选可用性和标注覆盖率。开发集、验证集和测试集分离，用于冻结阈值、prompt 配置和停止规则。

表 V 总结任务范围和官方轨迹规模。6,500 条官方轨迹来自LIBERO 每任务50 条轨迹的发布口径，用于说明benchmark 的数据规模和任务覆盖范围。

图 3 展示robosuite 环境背景，用于说明本文评测所依赖的MuJoCo 操作环境类型。

B. 总体性能比较

本节采用统一协议进行比较：相同130 个LIBERO 任务、相同感知和关键点配置、相同底层规划器，以及最多3 轮额外约束生成预算。表 VI 给出系统级结果，图 4 和图 5 分别展示总体成功率和任务组分布。

表 VI 联合报告自然任务与约束失配任务的成功率、检测质量、归因质量、修复收益、安全干预和成本。ReFlex 的自然任务成功率为80.5%，较原始ReKep 的50.6% 提高29.9 个百分点，较Code-as-Monitor 适配基线的70.2%

TABLE V
LIBERO 任务覆盖范围与官方轨迹规模。

任务组	任务数	占比	每任务 轨迹	轨迹 总数
LIBERO-Spatial	10	7.69%	50	500
LIBERO-Object	10	7.69%	50	500
LIBERO-Goal	10	7.69%	50	500
LIBERO-Long	10	7.69%	50	500
LIBERO-90	90	69.23%	50	4,500
Total	130	100%	—	6,500

占比以全部130 个任务为分母；由于四舍五入，各行占比之和可能不严格等于100%。轨迹数量来自LIBERO 官方发布口径（每任务50 条）。LIBERO-Long (libero_10) 和LIBERO-90 共同组成LIBERO-100 (100 个任务、5,000 条官方轨迹)，父套件不重复计入。将LIBERO-90 纳入评测任务不同于其在原始协议中的预训练角色。来源：LIBERO paper, Secs. 4.2 and 4.4 和official dataset page。

提高10.3 个百分点。约束失配评测中的成功率为83.4%，检测F1 为91.0%，归因准确率为86.0%，首次修复成功率为82.0%。这些结果从任务完成与阶段诊断两个层面支持结构化反馈的有效性。

图 4 展示不同评测范围下的总体成功率，并明确区分130 任务总体、原40 任务子集和LIBERO-90 扩展组。

图 5 进一步展示不同任务组上的性能覆盖面，并通过配对收益图区分新增成功、退化和净收益。配对比较在同一任务、同一初始状态和同一预算下进行；将退化单独列出，有助于判断修复收益是否伴随对原本成功任务的干扰。

C. 消融实验与计算效率

消融实验回答一个核心问题：ReFlex 的收益来自哪些组件。我们在固定诊断样本集合、固定检测门控、固定生成器和固定预算的条件下，逐个移除关键组件，包括数值证据、约束归因、RERS-v1 结构化组织和局部修复范围。图 6 展示相应消融结果。

消融比较同时考察修复成功率和受保护要求保持率：前者衡量错误要求是否得到修正，后者衡量原本正确的约束需求是否保留。图 6 中移除关键组件后的性能变化说明，数值证据、归因、结构化组织与局部编辑范围共同影响修复质量，不能仅用检测准确率代替闭环效果。

计算效率分析进一步考察性能收益与运行成本之间的关系。我们改变额外约束生成预算 $K \in \{0, 1, 2, 3, 5\}$ ，统计修复成功率、平均token 成本和每轮wall-clock time 组成。图 7 展示预算、成本和时延之间的关系。

图 7 从预算曲线、成功率-成本关系和时间分解三个视角分析效率。增加生成预算能够提供更多修复机会，同时增加token 消耗和时延；因此，性能收益需要结合完整rollout 的成本共同解释。表 VI 中ReFlex 的平均rollout 时间为17.1 s，低于Code-as-Monitor 适配基线的19.7 s 和PRoC3S 适配基线的21.3 s。

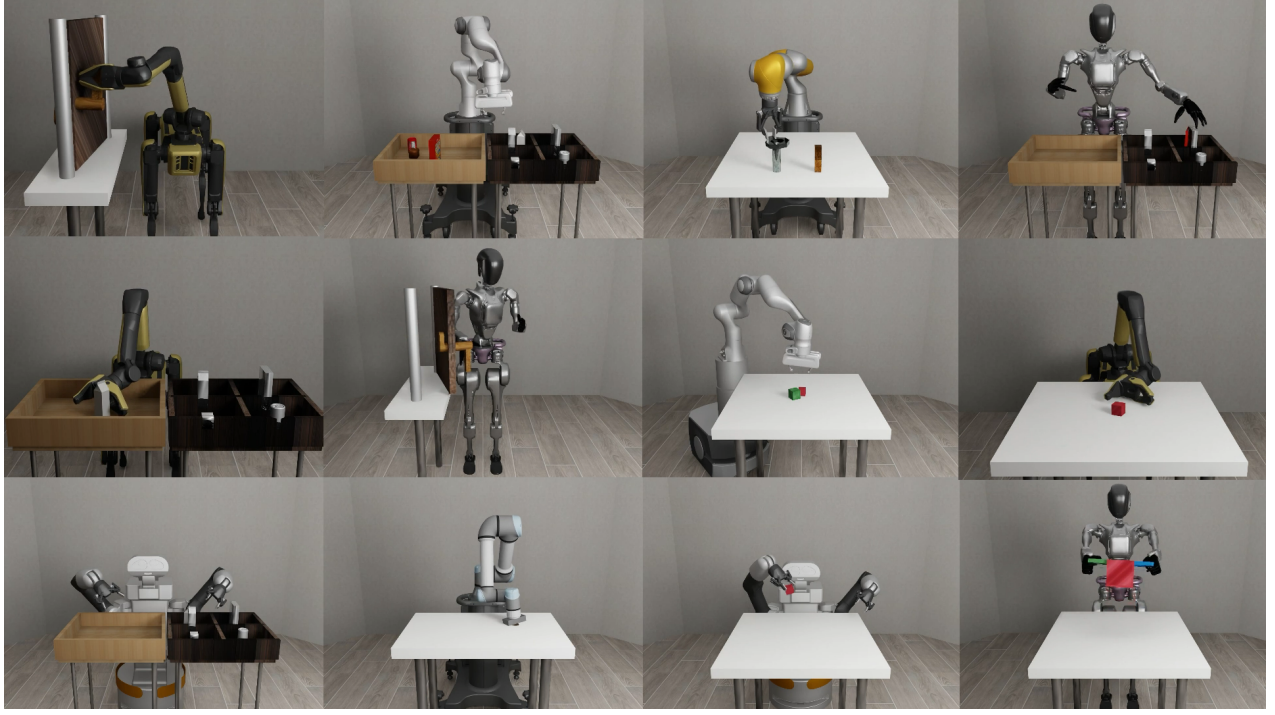


Fig. 3. 官方robosuite 环境概览图，用于说明MuJoCo 操作环境的多样性和本文实验设置背景。

TABLE VI
不同检测与修复机制的系统级实验结果。箭头表示指标的优选方向；未提供相应诊断接口的方法以横线标记不适用的指标。

方法	任务成功率(%)		检测(%)				归因与修复(%)		安全与成本	
	Nat. $\uparrow$	Mismatch $\uparrow$	F1 $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	False-blk. $\downarrow$	Attr. $\uparrow$	Repair@1 $\uparrow$	Harm. int. $\downarrow$	Time (s) $\downarrow$
ReKep (original)	50.6	46.2	—	—	—	—	—	—	—	12.4
ReKep + planner retry	53.1	48.7	—	—	—	—	—	—	—	14.8
ReKep + regeneration	58.4	54.9	—	—	—	—	—	—	—	16.2
ReKep + text feedback	62.7	60.1	62.0	71.0	55.0	6.8	48.0	41.0	3.1	18.5
ReKep + feasibility feedback	60.9	63.3	71.0	76.0	67.0	5.4	55.0	52.0	2.7	15.9
PRoC3S (adaptation)	68.1	71.5	78.0	81.0	75.0	4.2	63.0	61.0	2.0	21.3
Code-as-Monitor (adaptation)	70.2	73.8	82.0	84.0	80.0	3.8	70.0	68.0	1.8	19.7
ReFlex (ours)	80.5	83.4	91.0	90.0	92.0	1.6	86.0	82.0	0.9	17.1
Human-verified reference	91.0	95.0	98.0	98.0	98.0	0.4	97.0	95.0	0.2	manual

检测、归因和修复指标使用候选级（阶段）分母，任务成功率使用完整rollout 分母；两类总体不同，不能相乘解释为漏斗。

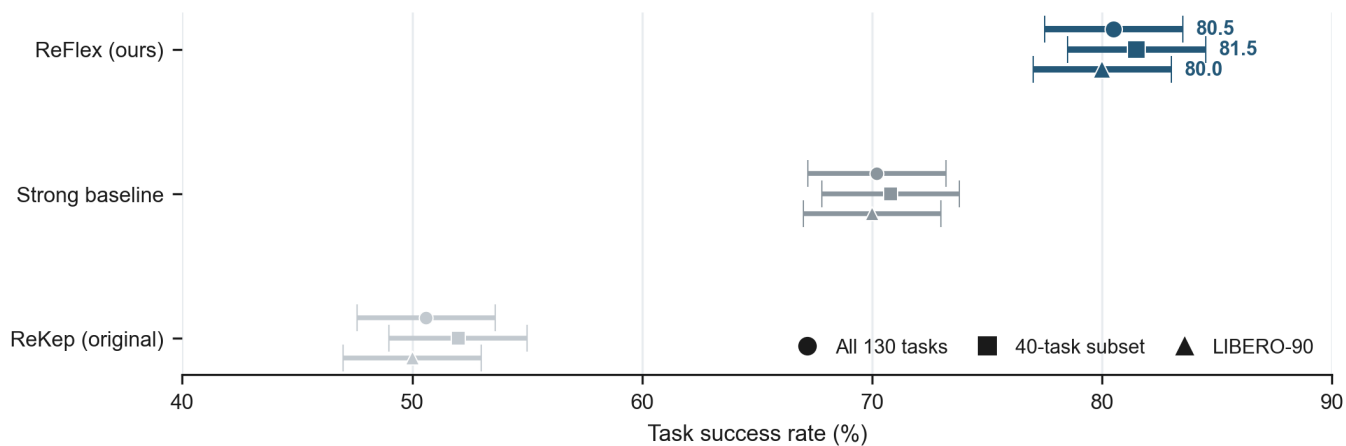


Fig. 4. 不同评测范围下的总体任务成功率。点表示成功率，横向误差线表示95% 置信区间；点形表示任务范围（圆形：全部130 任务，方形：40 任务子集，三角形：LIBERO-90），行表示三种方法。

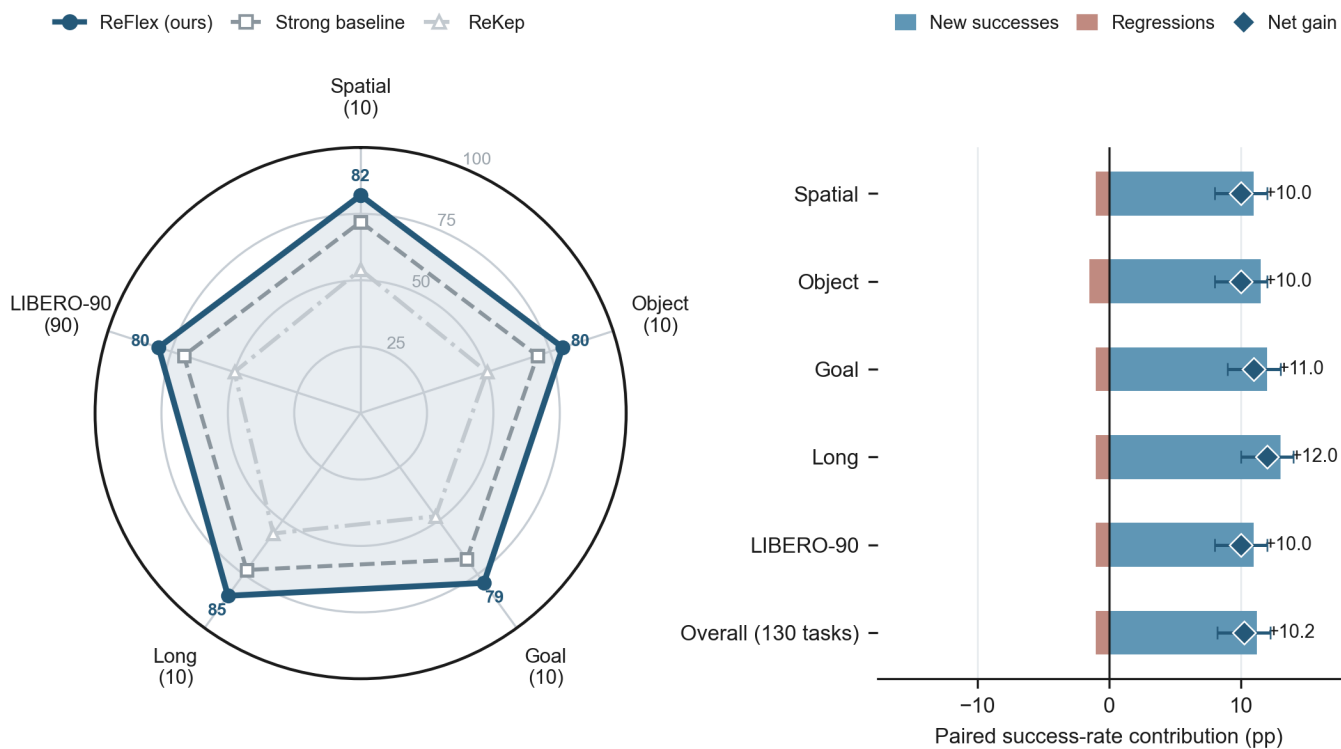


Fig. 5. 跨任务组性能轮廓与配对收益分解。左图为五个LIBERO 任务组上的成功率雷达图；右图为ReFlex 相对强基线的配对收益，包括新增成功、退化及净收益及95% 区间。

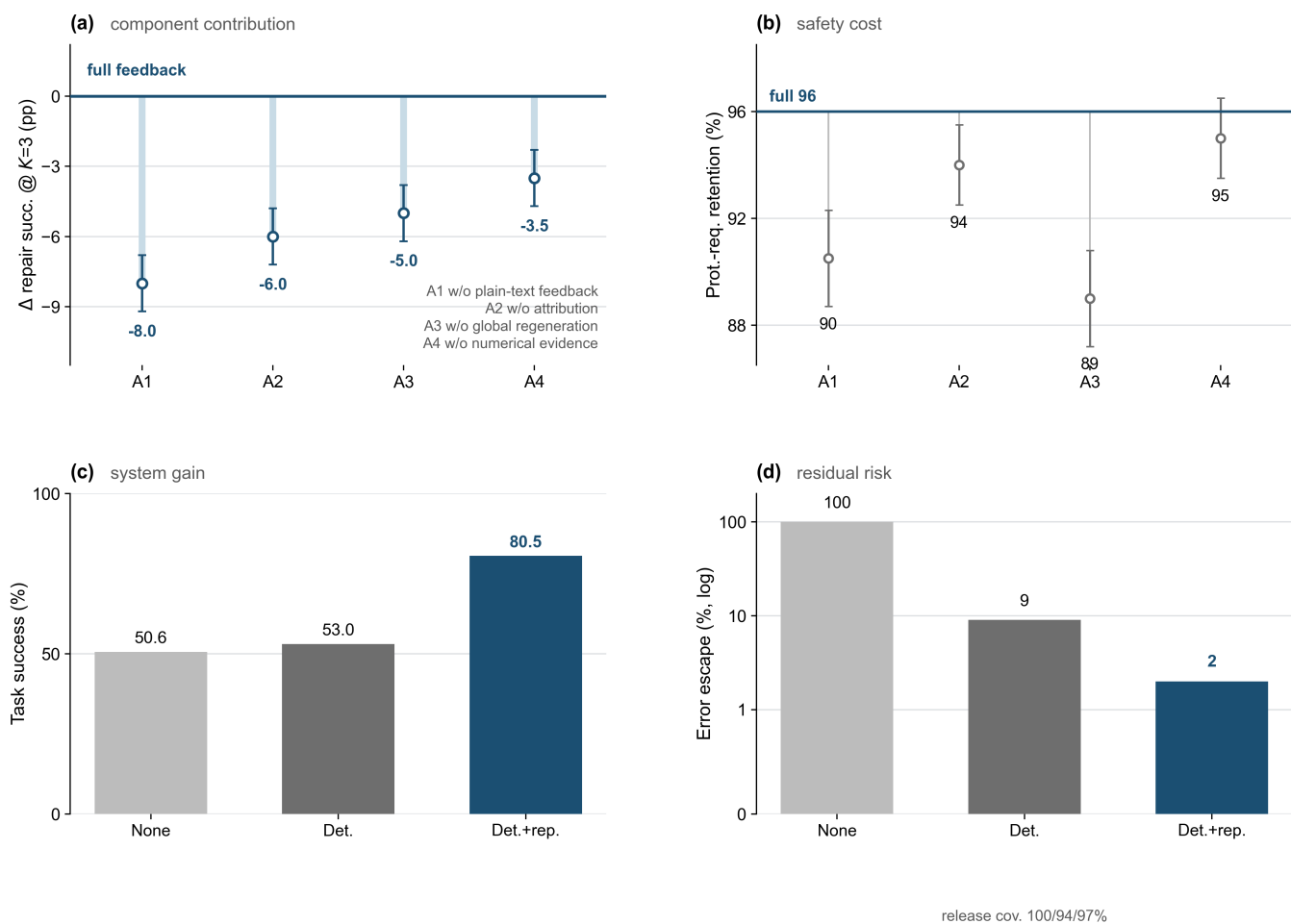


Fig. 6. 组件消融与系统级效果。(a) 在 $K = 3$ 下，相对完整反馈版本的修复成功率变化；负值表示移除组件后性能下降。(b) 受保护要求保持率，即原本正确的约束需求被保留的比例。(c) 原始管线、仅检测和检测加修复三种设置下的端到端任务成功率。(d) 对数坐标下的错误逃逸率，并标注放行覆盖率。

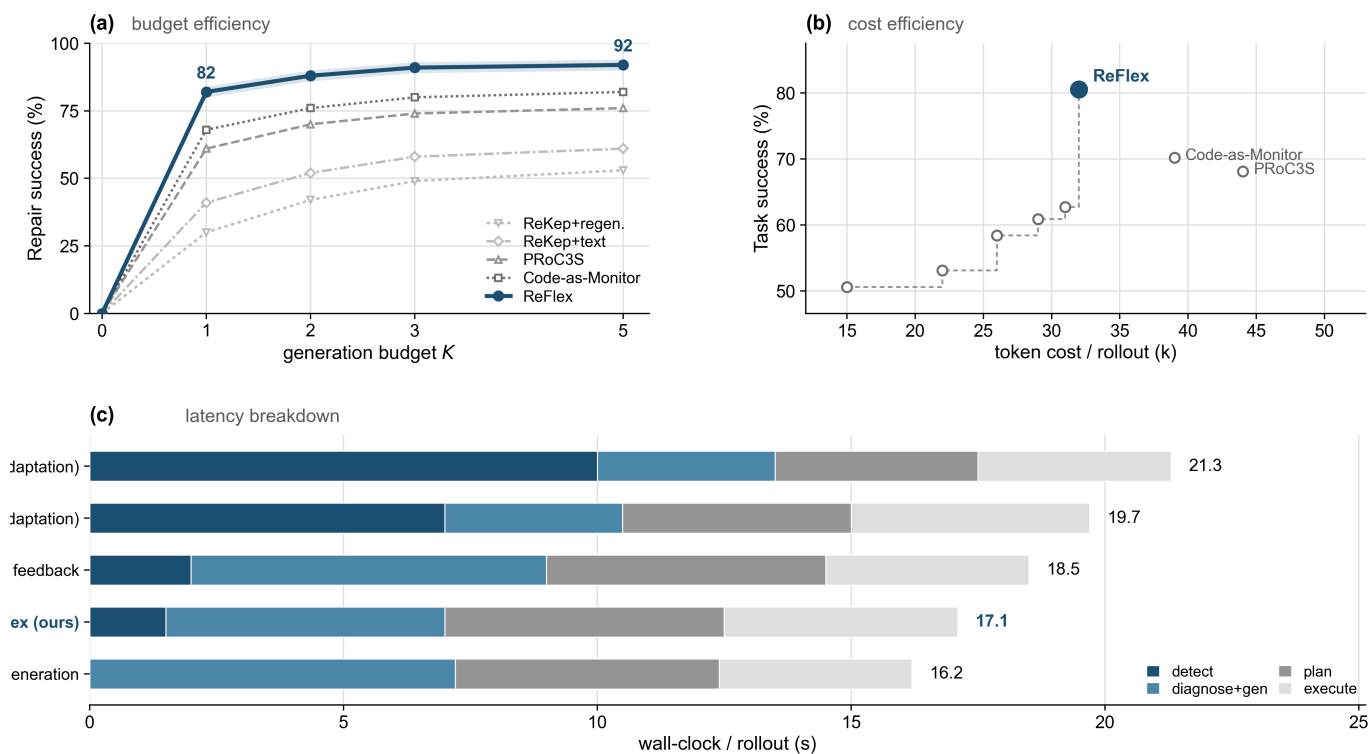


Fig. 7. 修复预算、token 成本与时延分析。(a) ReFlex 与四个反馈基线在不同额外生成预算 K 下的修复成功率；阴影表示 ReFlex 的 95% 区间。(b) 端到端任务成功率与平均 token 成本之间的关系，虚线表示 Pareto frontier。(c) 单次 rollout 的 wall-clock time 分解，包括检测、诊断加生成、规划和执行。

D. 经典错误场景的MuJoCo 定性对照

我们基于ReFlex 的五类错误体系，在四个独立MuJoCo 场景中展示正确与错误行为的定性对照：Scene A 展示对象绑定与进展停滞，Scene B 展示锅铲功能区域绑定，Scene C 展示按钮接近方向，Scene D 展示挡板处的碰撞风险。机器人使用Panda 模型，生活物体使用扫描网格或场景几何；每组共享物体布局、任务要求和相机。图 1 展示阶段关键帧，附录 A 给出对应的分阶段约束规格。

本节通过约束方程组与可见行为的对应关系，分析subgoal/path 中的引用内容、方向条件和必要条件缺失。正确列表示任务要求下的参考行为，错误列表示与任务要求不一致的行为；该对照用于说明约束层的修复目标，自动闭环性能由前述LIBERO 定量评测单独报告。

a) 相同对象与相同终点仍可能对应不同任务语义。：对象绑定组将抓取参考点从苹果替换为梨；功能区域组保留锅铲对象，却将手柄参考点替换为铲面参考点。两组错误动作都能平滑到达各自的抓取位置，但违反了独立任务要求。按钮组进一步保留相同TCP 终点、夹爪姿态与开度，只在最后接近段区分向下与水平运动。仅根据终点误差无法区分该组行为，方向证据必须关联到具体接近阶段。

b) 停滞需要时间证据与阶段完成条件。：图 8 比较相同抓取与抬升之后的搬运。正确参考的目标距离下降至放置参考点附近；错误参考存在运动，但搬运后段的距离没有持续缩小。对应的规格差异是式 (16) 中的终点要求，而非夹爪姿态或抓取保持。若将该现象组织为修复反馈，应明确失败阶段为搬运、证据窗口为到达前的局部往复，并要求补齐有效终点条件，同时保护原本正确的对象引用与抓取保持。终点遗漏只是一个候选解释，不能由演示反推出规划器必然停滞。

c) 安全约束应覆盖整条搬运路径与被持物体。：Scene D 的两组参考均指向相同放置目标。正确组升高后越过挡板，随后降低苹果、松爪并抬离；错误候选沿低位转运，被持苹果的几何会进入挡板。实际错误播放在接触前停止，图中的红色虚线表示未播放的危险后续路线。式 (19) 因而把安全间隙写入path，并同时考虑机器人与苹果几何；修复目标是补充避障条件，同时保留原任务的放置目标。

这些对照建立了从约束差异到可观察现象、再到候选局部修复要求的说明链条，并为前述定量指标提供了具体的任务语义解释。

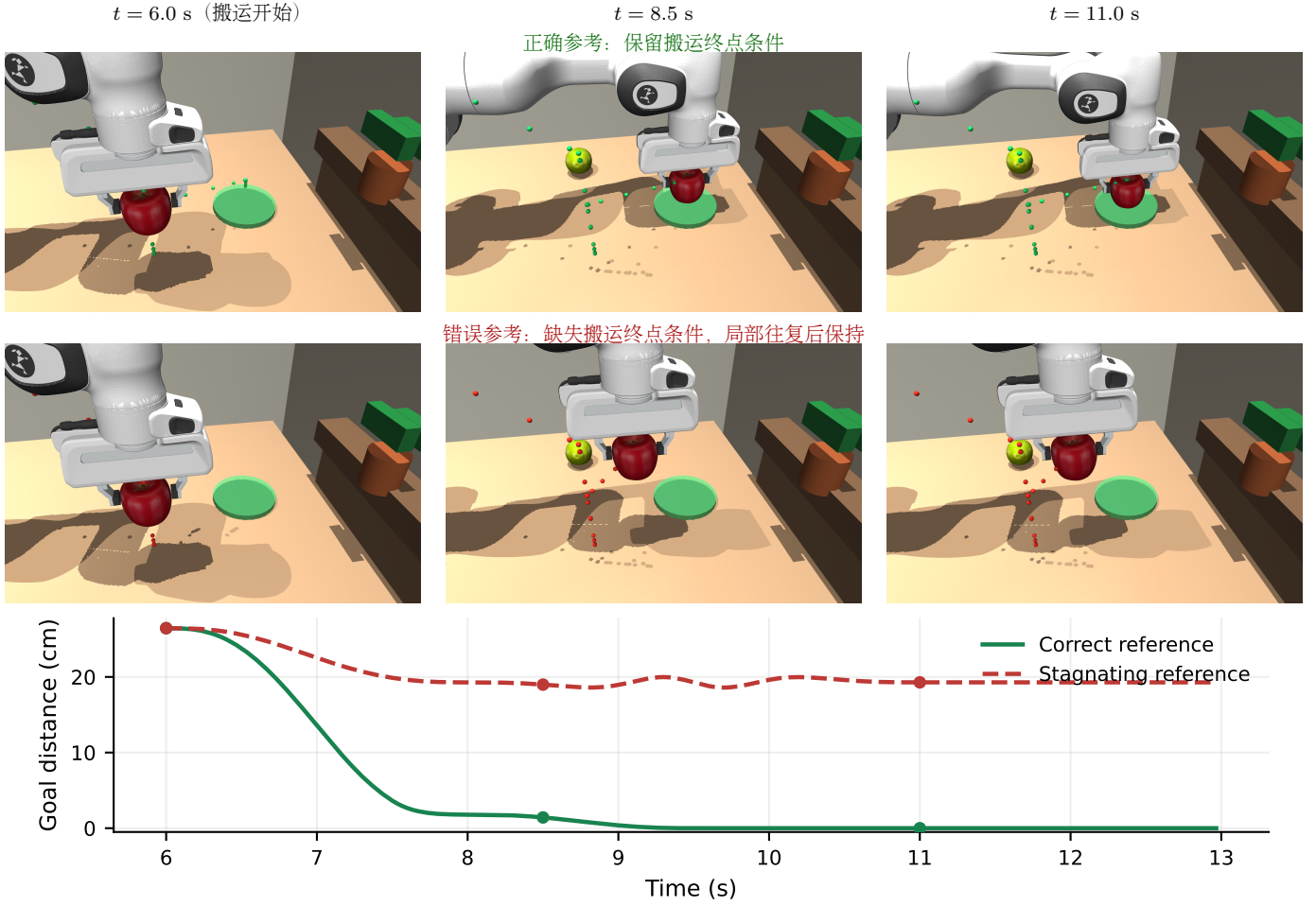


Fig. 8. Scene A 进展停滞案例的时序对照。距离曲线由TCP 坐标逐帧计算，目标为搬运阶段的放置参考点；点标记对应上方三个时刻。正确行为到达目标后仍保持夹持，错误行为在中途局部往复。曲线展示各自轨迹的时间过程，到达后的正常静止不计为停滞。

V. 讨论

当前版本的ReFlex 有意聚焦在一个窄但重要的失败范围：*constraint-induced motion planning errors*。这个边界既是方法论选择，也是工程上的现实选择。从方法论上看，它使我们能够单独分析ReKep 类系统中的关键接口问题，即VLM 生成的阶段约束可能看起来合理，但仍然不完整、过弱或与任务语义错位。从工程上看，这一边界提供了清晰的干预点：不需要立即重做感知模块、训练新规划器或修改底层控制器，而是在约束生成接口外增加一个反思式反馈层。

A. 当前边界

五类MuJoCo 定性对照具体化了约束规格差异，展示对象引用、功能区域、接近方向、阶段完成条件与路径安全要求的行为含义。定性对照用于解释候选编辑的范围，闭环性能由LIBERO 的检测、归因、修复与任务成功率指标评价。尤其是停滞不能只由静止帧判定，碰撞风险也不能仅依据TCP 是否绕开障碍判定。

这一边界必须明确。ReFlex 当前不试图覆盖所有具身操作失败来源。若目标物体没有被正确观测，场景表示严重错误，或下游优化器因数值原因失败，那么ReFlex 可

能仍能检测到轨迹症状，但不一定能够消除真正根因。本文的评测聚焦约束本身是主要错误来源的情形，考察结构化执行前反馈的诊断和修复效果；该结论不自动推广到其他失败来源。

B. 为什么约束接口值得修复

尽管范围有限，约束接口仍然是一个很有价值的修复对象。基础模型机器人系统的一个共同问题是：语义推理与几何执行之间通常依赖某种中间规格，而这个中间规格既强大又脆弱。ReKep 将这一接口显式化为阶段性的subgoal constraints 和path constraints。这种显式性正是ReFlex 能够工作的原因：一旦轨迹表现出症状，系统可以把症状转化为对约束规格的结构化批评，而不是只在下游拒绝执行。

C. 优势与潜在弱点

从设计上看，ReFlex 最可能在语义结构清晰、错误可局部定位的场景中发挥作用，例如目标对象绑定错误、功能区域绑定错误、趋近方向错误和阶段成功条件过弱。这些情况正是结构化报告和局部重生成相对自由文本提示更有优势的地方。相反，对于复杂接触动力学、抓取稳定性、物体滑落或感知缺陷引起的问题，当前ReFlex 可能

只能给出不完整诊断，需要与更强的几何推理、接触建模和在线执行恢复结合。

修复阶段本身也是当前系统最难的部分。检测只需要判断轨迹是否异常，归因需要解释异常更可能来自哪类约束缺陷，而修复则要求在不破坏原有正确约束的前提下改写约束。这正是RERS-v1 和局部prompt 约束的重要性：它们不仅组织诊断信息，也限制修复模型漂移、保护已有正确内容。本文结合修复成功率、受保护约束保持率和有害干预率分析这一权衡。

D. 后续方向

后续工作可以从几个方向展开。第一，扩展错误体系，将当前五类经典错误扩展到更复杂的时间顺序错误、接触丰富操作错误和物体间交互错误。第二，将归因从固定规则扩展为规则与学习式置信度校准结合的混合模型。第三，让修复闭环不仅能修改阶段约束，还能在必要时调整求解器参数、候选subgoal 或复检策略。第四，在本文评测基础上进一步考察未见布局、未见对象、不同生成模型和更多干扰物条件下的泛化能力。

更广义地说，ReFlex 可以与上游感知验证和下游执行恢复结合，形成更完整的具身反馈系统。当前版本是一个执行前反思层；未来系统可以进一步加入在线监测、回滚策略、失败记忆和长时程自适应重规划。本文当前阶段的价值在于提出并具体化这一中间层：让VLM 生成的任务规格不再被视为一次性固定输出，而是可以在执行前被检查、批评和修复的对象。

VI. 结论

本文提出ReFlex，一个面向ReKep 类机器人运动规划的反馈增强约束修复框架。与把规划失败仅视为下游执行问题不同，ReFlex 在规划与执行之间插入反思层：它检测经典轨迹症状，将其组织为结构化报告，归因到可能的约束缺陷，并把诊断结果反馈给阶段约束生成器，用于局部修复和重规划。

本文的核心思想是：在基础模型驱动的机器人操作系统中，很多失败并不应被简单理解为控制错误，而应被理解为连接语义推理与运动生成的中间规格错误。ReFlex 正是围绕这一接口展开。RERS-v1 报告格式、症状到归因的映射和受限修复prompt 共同服务于一个目标：让失败分析不仅能描述问题，还能直接转化为可执行的约束修复信号。

本文在130 个LIBERO 任务上评价检测质量、归因质量、修复收益与成本。ReFlex 的自然任务成功率达到80.5%，相比原始ReKep 提高29.9 个百分点；约束失配评测成功率为83.4%，检测F1 为91.0%。五类经典错误的成对约束方程组与MuJoCo 定性对照进一步说明对象引用、功能区域、接近方向、阶段完成条件和路径安全要求的差异，以及局部修复应修改和保留的内容。

ReFlex 将VLM 生成的阶段约束从一次性输出转化为可检查、可归因、可修复的中间对象。本文的系统比较与组件消融表明，将结构化诊断反馈接回约束生成接口，能够改善所评测任务中的诊断、修复和任务完成表现。

APPENDIX A

五类MuJoCo 案例的分阶段约束规格

本附录给出五类经典错误场景的正确与错误分阶段约束规格，为正文的MuJoCo 定性对照提供数学表达。各组在

同一场景中保持任务要求不变， C^- 表示错误规格；错误规格可满足，不等于任务要求得到满足。

A. 共同符号与阶段条件

阶段区间记为 $[t_0, T]$ ； q 为机器人关节配置， R_e 为夹爪旋转， w 为每根手指的开度， T_e 和 T_o 分别为TCP 与被持物体的世界位姿。 R_i^{ref} 是该阶段指定的姿态参考。共同运动条件记为

$$\begin{aligned} \mathcal{H}_i &= \{q_{\min} \leq q(t) \leq q_{\max}, \\ e_R(R_e(t), R_i^{\text{ref}}) &\leq \epsilon_R, \\ 0 &\leq w(t) \leq w_{\max}\}, \end{aligned} \quad (11)$$

其中 e_R 为旋转的测地角误差。接近阶段要求 $w = w_{\text{open}}$ ；闭合阶段在抓取位姿处把开度从 w_{open} 平滑变为相应物体/区域的 w_o 。闭合后的刚性保持条件记为

$$\mathcal{A}_o = \{w(t) = w_o, \quad T_e(t)^{-1}T_o(t) = A_o\}, \quad (12)$$

其中 A_o 为闭合完成时的相对位姿。该条件表达抓取后物体与TCP 的位姿保持关系；力闭合与接触稳定性属于另行评价的物理条件。 $\epsilon_p, \epsilon_R, \theta$ 是约束规格的容差，应与ReFlex 症状检测器的阈值区分。

B. 对象绑定：苹果与梨

设 k_o 是物体 o 的TCP 抓取参考点， $h_a > 0$ 为接近高度。接近、下降和闭合三个阶段的条件分别为

$$\begin{aligned} \mathcal{C}_{\text{hover,sg}}(o) &= \{\|p_e(T) - (k_o + h_a e_z)\| \leq \epsilon_p\}, \\ \mathcal{C}_{\text{descend,sg}}(o) &= \{\|p_e(T) - k_o\| \leq \epsilon_p\}, \\ \mathcal{C}_{\text{close,sg}}(o) &= \{\|p_e(T) - k_o\| \leq \epsilon_p, w(T) = w_o\}. \end{aligned} \quad (13)$$

各阶段的path 包含 $\mathcal{H}_i$ ；接近与下降时夹爪保持打开，闭合时保持抓取位姿。正确组令 $o = \text{apple}$ ，错误组令 $o = \text{pear}$ ，但独立任务仍为抓苹果。Scene A 的此组图展示闭合后的保持阶段，不包含搬运或释放；苹果与梨的闭合开度分别按扫描模型校准为每指38.1 mm 和33.2 mm。修复应改对象引用，而非改变场景里的物体身份。

C. 功能区域：锅铲手柄与铲面

令 r 为锅铲功能区域， k_r 为该区域的抓取参考点， z_r^0 为初始物体高度。接近和下降阶段复用上一组的参考点条件，但对对象始终固定为锅铲。闭合与抬升要求

$$\begin{aligned} \mathcal{C}_{\text{grasp,sg}}(r) &= \{\|p_e(T) - k_r\| \leq \epsilon_p, w(T) = w_r\}, \\ \mathcal{C}_{\text{lift,sg}} &= \{z_o(T) - z_r^0 \geq h_l\}, \\ \mathcal{C}_{\text{lift,path}}(r) &= \mathcal{H}_i \cup \mathcal{A}_{\text{spatula}}. \end{aligned} \quad (14)$$

正确组令 $r = \text{handle}$ ，错误组令 $r = \text{blade}$ ；两组均抬升约 $h_l = 75$ mm。开度根据所选区域厚度分别校准为每指10.5 mm 和33.3 mm。功能错误由任务明确要求手柄定义，不以铲面不能被夹起为前提。修复保留对象和抬升要求，只修订区域引用及与其对应的抓取几何。

D. 趋近方向：按钮上方与侧向

令 k_b 为按钮中心上方的共同TCP 目标， $\mathcal{I}_a$ 为最后趋近段， $n_\downarrow = (0, 0, -1)^\top$ 。仅在 $\|v_e(t)\| > \epsilon_v$ 的运动样本上施加方向条件：

$$\begin{aligned} \mathcal{C}_{\text{approach,sg}}^+ &= \mathcal{C}_{\text{approach,sg}}^- = \{\|p_e(T) - k_b\| \leq \epsilon_p\}, \\ \mathcal{C}_{\text{approach,path}}^+ &= \mathcal{H}_i \cup \{w = w_b, v_e \cdot n_\downarrow \geq \|v_e\| \cos \theta\}, \\ \mathcal{C}_{\text{approach,path}}^- &= \mathcal{H}_i \cup \{w = w_b, v_e \cdot n_\rightarrow \geq \|v_e\| \cos \theta\}. \end{aligned} \quad (15)$$

这里 $t \in \mathcal{I}_a$ ；方向要求不施加到从初始姿态重新定位的运动。Scene C 的 $k_b = (0.62, -0.03, 0.340)$ m, $w_b = 3$ mm；第120 帧两组到达相同关节姿态，图中选第90 帧表现方向差异。正确组随后展示 $s_b(T) = 8$ mm 的按钮行程和亮灯状态，错误组停在接触前。方向约束描述接近几何，按钮行程与亮灯则表示任务状态。

E. 进展停滞：搬运终点条件缺失

两组先完成相同的苹果抓取和抬升。对于之后的搬运阶段，令 k_G 是盘面附近的TCP 放置参考点， $d_G(t) = \|p_e(t) - k_G\|$ 。对照规格为

$$\begin{aligned} \mathcal{C}_{\text{carry,sg}}^+ &= \{d_G(T) \leq \epsilon_p\}, \\ \mathcal{C}_{\text{carry,path}}^+ &= \mathcal{H}_i \cup \mathcal{A}_{\text{apple}}, \\ \mathcal{C}_{\text{carry,sg}}^- &= \emptyset, \\ \mathcal{C}_{\text{carry,path}}^- &= \mathcal{H}_i \cup \mathcal{A}_{\text{apple}}. \end{aligned} \quad (16)$$

错误规格删除了有效的搬运阶段终点要求，只保留姿态和抓取保持，因此允许中途局部往复。图 8 将这一失配表现为目标距离不再有效缩小；终点条件缺失允许该行为，但不意味着它是唯一可能的轨迹。Scene A 此组正确行为完成搬运到放置参考位姿后仍保持夹持。

若诊断或修复使用进展要求，可以在事先声明的有效推进窗口 $\mathcal{I}_p$ 上考察

$$d_G(t + \Delta) - d_G(t) \leq -\eta, \quad t \in \mathcal{I}_p, \quad (17)$$

其中窗口排除正常等待、绕障和到达后的保持阶段，不要求所有轨迹点的欧氏距离单调下降。独立任务完成判据仍应由到达目标及相应阶段状态决定。

F. 碰撞风险：搬运路径安全条件缺失

先完成苹果抓取与初始抬升。定义目标区域的放置谓词

$$\begin{aligned} \Phi_G(T_o) &\iff (\|c_{o,xy} - g_{xy}\| \leq \epsilon_{xy}) \\ &\quad \wedge (|z_{\min}(T_o) - z_{\text{plate}}| \leq \epsilon_z), \end{aligned} \quad (18)$$

其中 $c_{o,xy}$ 是约定的物体放置参考中心， $z_{\min}(T_o)$ 是扫描网格最低点的世界高度。对搬运段， $\mathcal{B}(t) = \mathcal{R}(t) \cup \mathcal{A}(t)$ 包含机械臂、夹爪和被持苹果；与禁止接触挡板之间的规格为

$$\begin{aligned} \mathcal{C}_{\text{carry,sg}}^+ &= \mathcal{C}_{\text{carry,sg}}^- = \{\Phi_G(T_o(T))\}, \\ \mathcal{C}_{\text{carry,path}}^+ &= \mathcal{H}_i \cup \mathcal{A}_{\text{apple}} \cup \{\text{dist}(\mathcal{B}(t), \mathcal{O}) \geq \delta\}, \\ \mathcal{C}_{\text{carry,path}}^- &= \mathcal{H}_i \cup \mathcal{A}_{\text{apple}} \cup \{z_e(t) = z_{\text{low}}\}. \end{aligned} \quad (19)$$

低位高度条件只作用于经过挡板的转运段，之后仍需下降到相同的预期放置位姿。示例中 $z_{\text{low}} = 0.350$ m，正确跨越段的TCP 高度约0.490 m，所用间隙下界要求为 $\delta = 20$

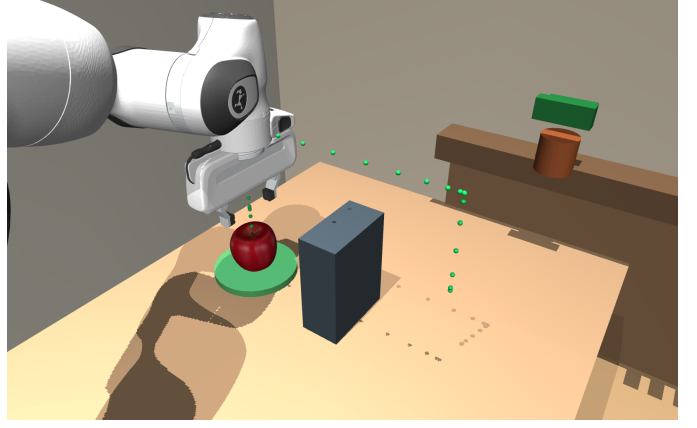


Fig. 9. Scene D 正确参考的最终状态：苹果位于盘面，夹爪已经松开并抬离，展示放置、释放和撤离阶段对应的目标状态。

mm。正确轨迹的苹果与挡板包围盒间隙下界约61 mm，机器人碰撞几何包围盒的分离下界不小于约34 mm；这些保守下界用于确认该参考动作分离，不冒充精确网格距离。

完整低位候选具有与正确组相同的预期放置位姿，但苹果网格顶点会进入挡板内部。错误播放只展示其接触前前缀，并在约2.5 mm 间隙处停止；它没有到达目标，不能被描述为已满足完整错误方程组的执行结果。红色虚线仅表示未播放的危险候选后续路线。

正确组的后续阶段还包括放稳、松爪和抬离。以 T_s 为放稳时刻，释放及抬离要求可写为

$$\begin{aligned} \mathcal{C}_{\text{release,sg}} &= \{w(T) = w_{\text{open}}, \Phi_G(T_o(T))\}, \\ \mathcal{C}_{\text{release,path}} &= \mathcal{H}_i \cup \{T_o(t) = T_o(T_s)\}, \\ \mathcal{C}_{\text{retreat,sg}} &= \{z_e(T) - z_e(T_s) \geq h_r\}, \\ \mathcal{C}_{\text{retreat,path}} &= \mathcal{H}_i \cup \{w = w_{\text{open}}, T_o(t) = T_o(T_s)\}. \end{aligned} \quad (20)$$

Scene D 第405 帧开始松爪，第435 帧开始抬离约 $h_r = 69$ mm；放置后苹果位姿保持，网格最低点与盘面齐平。整个过程为540 帧、30 fps。图 9 展示正确组的最终状态，明确区分到达目标上方与完成放置。

APPENDIX B

用于约束修复的完整PROMPT 包

本附录给出当前ReFlex 框架中用于约束修复的prompt 包。当前系统中，prompt-conditioned generation 主要出现在repair 阶段；前面的检测、报告和规则归因阶段仍以确定性规则为主。因此，本附录重点说明RERS-v1 报告和约束归因完成后，如何构造用于阶段约束重生成的修复prompt。

A. 标准Prompt 模板

ReFlex 使用结构化prompt 包，而不是单一自由文本prompt。下面列出的字段会由当前失败规划实例中的阶段上下文、原始约束和RERS-v1 报告内容填充。

- 1) **Role:** “You repair stage-level robot constraints for ReKep-style motion planning. You must perform conservative local edits rather than rewriting the whole task.”

- 2) **Task:** “Revise the current failed stage constraints using the structured ReFlex report.”
- 3) **Authoritative Inputs:** 任务指令、轨迹ID、报告ID、当前阶段ID 和当前阶段名称。
- 4) **Repair Objective:** 只修复当前失败阶段的约束，使其解决报告中的症状并覆盖归因原因。
- 5) **Original Constraints:** 当前失败阶段的完整约束bundle。
- 6) **Original Stage Subgoal Constraints:** 当前阶段原始subgoal constraint 文本。
- 7) **Original Stage Path Constraints:** 当前阶段原始path constraint 文本。
- 8) **Symptom Events:** 序列化后的RERS-v1 症状事件，包括类型、严重程度、置信度、时间范围、规则ID、解释和证据特征。
- 9) **Constraint Attributions:** 序列化后的归因项，包括原因类型、目标约束范围、置信度、诊断理由和修复指令。
- 10) **Must Preserve:** 明确禁止重写无关阶段、改变正确内容或引入新任务目标的保留规则。
- 11) **Must Modify:** 从归因结果得到的目标编辑要求，通常只作用于当前失败阶段。
- 12) **Few-shot Repair Patterns:** 针对代表性语义、几何和碰撞类错误的简短修复示例。
- 13) **Verification Checklist:** 后置自检要求，确保修复输出覆盖所有归因原因，同时保留无关任务内容。
- 14) **Output Contract:** schema-constrained JSON 输出规范，分离修复后的subgoal constraints、path constraints 和self-check 语句。

该prompt 包刻意保持局部、归因驱动和schema 约束。模型不是被要求重新设计整个任务，而是在明确保留规则和验证规则下，保守修改当前失败阶段。

B. Few-Shot 修复模式

下面按五类症状组织论文中的before/after 修复模式，并与附录 A 的方程组对应。其中阶段完成条件模式补齐进展停滞案例的修复说明；这些文本是修复模板，不是MuJoCo 演示实际执行的自动修复记录。

Object binding repair

Mapped cause: target_object_description_mismatch.
Before: “Move the end-effector toward the nearest elongated object and prepare to grasp it.”

After: “Move the end-effector toward the task-intended object specified by the user instruction, explicitly grounding the approach to the target object rather than a nearby distractor.”

Keypoint / region repair

Mapped cause: target_keypoint_or_region_binding_mismatch.

Before: “Reach the target object and align with a convenient point on its surface.”

After: “Reach the target object and align the end-effector with the task-relevant keypoint or functional region required by this stage.”

Approach direction repair

Mapped cause: approach_direction_or_relative_geo-

metry_mismatch.

Before: “Approach the target while maintaining a feasible pose.”

After: “Approach the target from the intended direction and enforce the correct relative geometric relation needed for successful stage completion.”

Progress / stage completion repair

Mapped cause: stage_success_condition_too_weak.

Before: “Keep the apple grasped and maintain the required gripper pose during transport.

After: “Preserve the apple binding, grasp attachment and gripper pose; add an explicit transport-stage endpoint condition at the intended placement reference. Interpret progress over the active transport window, excluding normal waits, detours and completed-stage holds.

Collision repair

Mapped cause: missing_collision_avoidance_or_clearance_requirement.

Before: “Move to the target pose as quickly as possible.”

After: “Move to the target pose while explicitly maintaining obstacle avoidance and a safe clearance margin throughout the path.”

C. 验证清单与输出契约

修复输出必须通过以下轻量检查：

- 1) 只修改当前失败阶段，不重写无关阶段。
- 2) 保留所有与报告错误无关的正确约束内容。
- 3) 修复后的约束必须显式覆盖报告中的每个归因原因。
- 4) 不引入原始任务中不存在的新目标、新对象或新操作意图。

最终模型输出需要满足以下结构化契约：

repair_target (string)

当前修复的主要约束原因。

edited_constraint_scope (string)

本次编辑作用于当前阶段约束的哪一部分。

repair_rationale (string)

简短说明本次编辑如何处理归因原因。

keep_unchanged (list)

明确保留的约束内容。

repaired_subgoal_constraints (string)

修复后的当前阶段subgoal constraints；若不需要修改则返回原内容。

repaired_path_constraints (string)

修复后的当前阶段path constraints；若不需要修改则返回原内容。

self_check (list)

确认输出满足验证清单的自检语句。

上述prompt 模板、few-shot 模式、验证规则和输出字段共同构成当前ReFlex repair 阶段的完整prompt package。